

Milestone 3

(Sprint 1)

Team 50

Software Engineering Project: Jan-Setu



Team Members

Name	Email
Tushar Sharma	21f3000007@ds.study.iitm.ac.in
Amit Kumar Gupta	21f1005763@ds.study.iitm.ac.in
Sarthak Kumar Tiwari	23f3000340@ds.study.iitm.ac.in
Parkhi Yadav	22f3002870@ds.study.iitm.ac.in
Veer Vikram Singh	23f3000375@ds.study.iitm.ac.in

Jan Setu — API Test Report

1. Executive Summary

Metric	Value
Test Framework	pytest 8.x with FastAPI TestClient
API Operations Discovered	29 across 6 functional groups
Direct HTTP API Tests	8 (full request→response)
Unit / Contract Tests	106 (mocked I/O, pure logic)
Test Modules	12 source files
Warnings	1 (Starlette deprecation)
Database Required	No (all tests use mocked sessions)
External Services	None (WhatsApp, STT, LLMs mocked)

2. Test Environment & Config

Parameter	Value
Operating System	macOS (Apple Silicon)
Python Version	3.11
Web Framework	FastAPI + Starlette
ORM / DB Driver	SQLAlchemy 2.x + asyncpg
Test Client	<code>fastapi.testclient.TestClient</code>
Mocking Strategy	<code>monkeypatch</code> , in-memory fakes

Test command executed:

```
uv run pytest tests/ -v --tb=short
```

3. Test Results by Module

#	Module	Source File	Tests	Pass	Fail
1	WhatsApp Webhook & API	<code>test_webhook.py</code>	14	✓ 14	0
2	Citizen Authentication	<code>test_auth.py</code>	10	✓ 10	0
3	Classification & LLM Pipeline	<code>test_classify.py</code>	17	✓ 17	0
4	Application Configuration	<code>test_config.py</code>	7	✓ 7	0
5	Conversation State Machine	<code>test_conversation.py</code>	18	✓ 18	0
6	Deduplication Engine	<code>test_dedup.py</code>	3	✓ 3	0
7	Logging & Observability	<code>test_logging.py</code>	2	✓ 2	0
8	Media Handling	<code>test_media.py</code>	6	✓ 6	0
9	Officials Console	<code>test_officials.py</code>	5	✓ 5	0
10	PDF Generation	<code>test_pdf.py</code>	6	✓ 6	0
11	Taxonomy & Routing Policy	<code>test_taxonomy.py</code>	7	✓ 7	0
12	Webhook Events Repository	<code>test_webhook_events_repository.py</code>	3	✓ 3	0
TOTAL			114	✓ 114	✗ 0

Development Defect Identification & Resolution Matrix

💡

Value Demonstration — Caught Discrepancies During Active Development

Testing identified critical implementation defects and security edge cases during initial development. Below are **6 test cases** that initially **FAILED** due to discrepancies between expected and actual output. Following root-cause diagnosis, application logic was patched. All 6 tests are now verified as **PASSED** in the final suite run (114/114 passing).

TC ID	API / TARGET COMPONENT	INITIAL DEV	EXPECTED OUTPUT	INITIAL (FAILING) OUTPUT	ROOT CAUSE & RESOLUTION	CURRENT RUN
TC-006	POST <code>/whatsapp/webhook</code>		<code>503 Service Unavailable</code> with secret error detail	<code>400 Bad Request ({"detail": "Invalid JSON payload"})</code>	Middleware lacked early production check before body parse. Reordered validation steps in <code>jan_setu/whatsapp/api.py</code> .	
TC-019	POST <code>/auth/request-code</code>		Normalized Indian ID: <code>"917652064884"</code>	Unformatted string: <code>" +91 76520 64884"</code>	Phone parser retained spaces & <code>+</code> symbol, breaking WhatsApp Meta contract. Added regex digit filter in <code>normalize_phone()</code> .	
TC-032	POST <code>/api/grievances/draft</code>		Confidence score clamped: <code>1.0</code>	Overflow confidence: <code>1.5</code>	LLM output returned confidence > 1.0, triggering downstream schema crash. Added <code>[0.0, 1.0]</code> clamp guard in <code>parse_classification()</code> .	
TC-053	Security Settings		Raises <code>ValueError</code> ("at least 32 bytes")	Settings allowed 9-byte secret (<code>"too-short"</code>)	Config class lacked length validator on JWT secret. Added 32-byte minimum secret validation in <code>jan_setu/config.py</code> .	
TC-061	WhatsApp Conversation		State remains <code>CONFIRMING_LOCATION</code>	State advanced to <code>AWAITING_ISSUE</code> on stale button click	State machine accepted button press from prior interaction. Added contextual token comparison in <code>advance()</code> .	
TC-091	GET <code>/api/official/queue</code>		<code>official_can_access()</code> returns <code>False</code> for cross-jurisdiction	Returned <code>True</code> (granted cross-jurisdiction access)	RBAC evaluator checked department without validating jurisdiction boundary. Patched scope check in <code>jan_setu/officials.py</code> .	

TC-001 Webhook Verification — Valid Token

✓ PASS

FIELD	DETAIL
API Tested	GET /whatsapp/webhook
Inputs	Query: hub.mode=subscribe, hub.verify_token=test_verify_token, hub.challenge=challenge-value ; Env: WHATSAPP_VERIFY_TOKEN=test_verify_token
Expected Output	200 OK ; Response body is the raw challenge string challenge-value (plain text)
Actual Output	200 OK ; Response text: challenge-value
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_WEBHOOK.PY:31-44

```
def test_webhook_verification_success(monkeypatch):
    monkeypatch.setenv("WHATSAPP_VERIFY_TOKEN", "test_verify_token")
    with TestClient(app) as client:
        response = client.get(
            "/whatsapp/webhook",
            params={
                "hub.mode": "subscribe",
                "hub.verify_token": "test_verify_token",
                "hub.challenge": "challenge-value",
            },
        )

    assert response.status_code == 200
    assert response.text == "challenge-value"
```

TC-002 Webhook Verification — Invalid Token Rejected

✓ PASS

FIELD	DETAIL
API Tested	GET /whatsapp/webhook
Inputs	Query: hub.verify_token=wrong-token (mismatched); Env: WHATSAPP_VERIFY_TOKEN=test_verify_token
Expected Output	403 Forbidden — token mismatch must be rejected
Actual Output	403 Forbidden ; {"detail": "Webhook verification failed"}
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_WEBHOOK.PY:47-59

```
def test_webhook_verification_rejects_bad_token(monkeypatch):
    monkeypatch.setenv("WHATSAPP_VERIFY_TOKEN", "test_verify_token")
    with TestClient(app) as client:
        response = client.get(
            "/whatsapp/webhook",
            params={
                "hub.mode": "subscribe",
                "hub.verify_token": "wrong-token",
                "hub.challenge": "challenge-value",
            },
        )

    assert response.status_code == 403
```

TC-006 Webhook POST — Requires App Secret in Production		✓ PASS
FIELD	DETAIL	
API Tested	POST /whatsapp/webhook	
Inputs	Env: ENVIRONMENT=production , WHATSAPP_APP_SECRET="" ; JSON: {"entry": []}	
Expected Output	503 Service Unavailable — app secret mandatory in production	
Actual Output	503 ; {"detail": "WHATSAPP_APP_SECRET is required outside development."}	
Result	✓ Success	
📦 PYTEST FUNCTION SNAPSHOT — TEST_WEBHOOK.PY:128-135		
<pre>def test_webhook_post_requires_secret_outside_development(monkeypatch): monkeypatch.setenv("ENVIRONMENT", "production") monkeypatch.setenv("JWT_SECRET", PRODUCTION_JWT_SECRET) monkeypatch.setenv("WHATSAPP_APP_SECRET", "") with TestClient(app) as client: response = client.post("/whatsapp/webhook", json={"entry": []}) assert response.status_code == 503</pre>		

TC-012 API Key — Missing Key Rejected in Production		✓ PASS
FIELD	DETAIL	
API Tested	GET /v1/contacts	
Inputs	Env: ENVIRONMENT=production, API_KEY=secret-key; No X-API-Key header sent	
Expected Output	401 Unauthorized — must not reach data layer	
Actual Output	401 Unauthorized; {"detail": "Invalid API key"}	
Result	✓ Success	
📦 PYTEST FUNCTION SNAPSHOT — TEST_WEBHOOK.PY:293-300		
<pre>def test_v1_endpoint_rejects_missing_api_key_outside_development(monkeypatch): monkeypatch.setenv("ENVIRONMENT", "production") monkeypatch.setenv("JWT_SECRET", PRODUCTION_JWT_SECRET) monkeypatch.setenv("API_KEY", "secret-key") with TestClient(app) as client: response = client.get("/v1/contacts") assert response.status_code == 401</pre>		

 **Note:**

Module 1 contains 14 total tests. The remaining 10 tests (TC-003 through TC-005, TC-007 through TC-011, TC-013, TC-014) cover HMAC signature verification, contact upsert SQL contracts, WhatsApp Cloud client injection, invalid/non-object JSON rejection, duplicate message handling, media/text/location/interactive message parsing, and API key configuration enforcement. All pass. Full details are in the companion Markdown report.

TC-019 Phone Normalization — Indian WhatsApp ID

✓ PASS

FIELD	DETAIL
API Tested	normalize_phone() — used by POST /auth/request-code
Inputs	"7652064884", "+91 76520 64884", "917652064884" (3 parametrized formats)
Expected Output	All normalize to "917652064884" (Meta Indian WhatsApp ID)
Actual Output	All 3 cases return "917652064884"
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_AUTH.PY:42-51

```
@pytest.mark.parametrize(
    ("raw_phone", "expected"),
    [
        ("7652064884", "917652064884"),
        ("91 76520 64884", "917652064884"),
        ("917652064884", "917652064884"),
    ],
)
def test_normalize_phone_uses_meta_indian_whatsapp_id(raw_phone, expected):
    assert normalize_phone(raw_phone) == expected
```

TC-020 JWT Access Token — Round-Trip Encode/Decode

✓ PASS

FIELD	DETAIL
API Tested	create_access_token() / decode_access_token() — issued by GET /auth/status
Inputs	Settings with test secret; user_id="user-123"
Expected Output	decode_access_token(token) returns "user-123"
Actual Output	"user-123" — round-trip successful
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_AUTH.PY:58-62

```
def test_create_and_decode_access_token_round_trips_user_id():
    settings = _settings()
    token = create_access_token(settings, user_id="user-123")

    assert decode_access_token(settings, token) == "user-123"
```

Note:

Module 2 contains 10 total tests covering: code shape validation, hash determinism/insensitivity, phone normalization (3 parametrized), JWT round-trip, wrong-secret rejection, login approval button round-trip, malformed ID rejection (4 parametrized), and browser label sanitization. All pass. Full details in the Markdown report.

TC-038 Structured Extraction — Multilingual (Hindi + English)

✓ PASS

FIELD	DETAIL
API Tested	<code>parse_extraction()</code> — v2 structured extraction pipeline
Inputs	Full extraction JSON with Hindi summary <code>"मुख्य सड़क पर खुला मैनहोल है"</code> , <code>category_id="open_or_damaged_manhole"</code> , <code>safety="immediate"</code>
Expected Output	All fields parsed including Unicode Hindi; <code>landmark="बस स्टैंड"</code>
Actual Output	All assertions pass — multilingual extraction works correctly
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_CLASSIFY.PY:127-151

```
def test_parse_structured_multilingual_extraction():
    raw = """{
        "summary": "मुख्य सड़क पर खुला मैनहोल है",
        "category_id": "open_or_damaged_manhole",
        "confidence": 0.94,
        "asset_scope": "public",
        "safety": "immediate",
        "landmark": "बस स्टैंड"
    }"""
    result = parse_extraction(raw)
    assert result is not None
    assert result.category_id == "open_or_damaged_manhole"
    assert result.safety == "immediate"
    assert result.asset_scope == "public"
    assert result.landmark == "बस स्टैंड"
```

TC-044 LLM Provider Chain — Groq First, OpenRouter Failsafe

✓ PASS

FIELD	DETAIL
API Tested	<code>build_extraction_attempts()</code> — LLM provider chain configuration
Inputs	Settings with <code>groq_api_key="gk"</code> , <code>openrouter_api_key="ok"</code>
Expected Output	4 attempts: <code>["groq", "groq", "groq", "openrouter"]</code> ; primary model <code>"openai/gpt-oss-120b"</code> with strict JSON schema
Actual Output	Provider chain and model configuration match expected values
Result	✓ Success

PYTEST FUNCTION SNAPSHOT — TEST_CLASSIFY.PY:271-285

```
def test_attempts_groq_first_then_openrouter_failsafe():
    from jan_setu.pipeline.classify import EXTRACTION_SCHEMA, build_extraction_attempts

    settings = _settings(groq_api_key="gk", openrouter_api_key="ok")
    attempts = build_extraction_attempts(settings, schema=EXTRACTION_SCHEMA)

    providers = [attempt["provider"] for attempt in attempts]
    assert providers == ["groq", "groq", "groq", "openrouter"]
    primary = attempts[0]["body"]
    assert primary["model"] == "openai/gpt-oss-120b"
    assert primary["reasoning_effort"] == "low"
    assert primary["response_format"]["type"] == "json_schema"
```

Note:

Module 3 contains 17 total tests covering: classification parsing (valid JSON, non-JSON, unknown category, missing/invalid fields, code fence extraction), image match parsing (valid, missing keys, confidence clamping), structured extraction (multilingual, unknown taxonomy, untrusted enum normalization), text+voice combining, STT integration, and LLM provider chain configuration (5 tests). All pass.

Modules 4–12 — Compact Summary with Representative Test Cases

For brevity in the printed version, modules 4–12 are shown with key representative test cases. The full Markdown report contains every test case with complete details.

4

Application Configuration

tests/test_config.py · 7 tests · Database URL construction, JWT security, environment validation

7
TESTS PASSED

TC-053 JWT Secret — Short Secret Rejected

✓ PASS

FIELD	DETAIL
API Tested	Settings validation (security guard)
Inputs	jwt_secret="too-short"
Expected Output	Raises ValueError with "at least 32 bytes"
Actual Output	ValueError raised with matching message
Result	✓ Success

📦 PYTEST FUNCTION SNAPSHOT — TEST_CONFIG.PY:51-55

```
def test_short_jwt_secret_is_rejected():
    with pytest.raises(ValueError, match="at least 32 bytes"):
        Settings(_env_file=None, jwt_secret="too-short")
```

5

Conversation State Machine

tests/test_conversation.py · 18 tests · Full WhatsApp conversation flow testing

18
TESTS PASSED

TC-056 New Contact — Greets and Requests Location

✓ PASS

FIELD	DETAIL
API Tested	advance() state machine — initial contact
Inputs	wa_id="911234567890", state=None, inbound text "hi"
Expected Output	State → AWAITING_LOCATION; single location_request_message; contains "नमस्ते"
Actual Output	State transition and message content correct
Result	✓ Success

📦 PYTEST FUNCTION SNAPSHOT — TEST_CONVERSATION.PY:51-61

```
def test_new_contact_greets_and_requests_location():
    result = advance(wa_id="911234567890", state=None, context=None, inbound=_inbound())

    assert result.state == STATE_AWAITING_LOCATION
    assert len(result.intents) == 1
    intent = result.intents[0]
    assert intent.payload["interactive"]["type"] == "location_request_message"
    assert "नमस्ते" in intent.text_body
    assert "location" in intent.text_body.lower()
```


9

Officials Console

tests/test_officials.py · 5 tests · JWT audience, access control, jurisdiction scoping

5
TESTS PASSED

TC-092 Department Officer — Limited to Assigned Departments

✓ PASS

FIELD	DETAIL
API Tested	official_can_access() — RBAC authorization
Inputs	Officer with department_keys=["sanitation"]; test "sanitation" and "public_works"
Expected Output	sanitation → True; public_works → False
Actual Output	Access correctly scoped to assigned department
Result	✓ Success
PYTEST FUNCTION SNAPSHOT — TEST_OFFICIALS.PY:50-53	
<pre>def test_department_officer_is_limited_to_assigned_departments(): official = _official(role="department_officer", department_keys=["sanitation"]) assert official_can_access(official, _grievance(department_key="sanitation")) assert not official_can_access(official, _grievance(department_key="public_works"))</pre>	

11

Taxonomy & Routing Policy

tests/test_taxonomy.py · 7 tests · Category resolution, safety routing, dispatch decisions

7
TESTS PASSED

TC-102 Immediate Safety — Emergency Redirect, No Auto-Dispatch

✓ PASS

FIELD	DETAIL
API Tested	evaluate_routing_policy() — safety-critical routing
Inputs	category="open_or_damaged_manhole", safety="immediate", confidence=0.99
Expected Output	disposition="emergency_redirect", priority="priority", dedup_eligible=False, needs_official_review=True
Actual Output	All routing decisions match safety-critical requirements
Result	✓ Success
PYTEST FUNCTION SNAPSHOT — TEST_TAXONOMY.PY:24-34	
<pre>def test_immediate_safety_never_auto_dispatches_or_deduplicates(): decision = evaluate_routing_policy(category_key="open_or_damaged_manhole", safety="immediate", asset_scope="public", extraction_confidence=0.99,) assert decision.disposition == "emergency_redirect" assert decision.priority == "priority" assert not decision.dedup_eligible assert decision.needs_official_review</pre>	

Complete API Endpoint Inventory

#	GROUP	METHOD	ROUTE	HTTP TEST	UNIT COVERAGE
1	Health	GET	/health	✓	✓
2	Health	GET	/ready	—	—
3	WhatsApp	GET	/v1/contacts	✓	✓
4	WhatsApp	GET	/v1/messages	—	—
5	WhatsApp	POST	/v1/messages/text	—	✓
6	Webhook	GET	/whatsapp/webhook	✓	✓
7	Webhook	POST	/whatsapp/webhook	✓	✓
8	Auth	POST	/auth/request-code	—	✓

#	GROUP	METHOD	ROUTE	HTTP TEST	UNIT COVERAGE
9	Auth	GET	/auth/status	—	✓
10	Auth	POST	/auth/approval-status	—	✓
11	Auth	POST	/auth/refresh	—	✓
12	Auth	POST	/auth/logout	—	✓
13	Grievances	POST	/api/grievances/transcription-preview	—	✓
14	Grievances	POST	/api/grievances/draft	—	✓
15	Grievances	GET	/api/grievances/{id}/draft	—	—
16	Grievances	PATCH	/api/grievances/{id}/review	—	✓
17	Grievances	PATCH	/api/grievances/{id}/photo	—	✓
18	Grievances	POST	/api/grievances/{id}/confirm	—	✓
19	Grievances	GET	/api/grievances	—	—
20	Grievances	GET	/api/grievances/{id}	—	—
21	Grievances	GET	/api/grievances/{id}/audio/{idx}	—	✓
22	Grievances	GET	/api/grievances/{id}/pdf	—	✓
23	Officials	POST	/api/official/auth/request-code	—	✓
24	Officials	POST	/api/official/auth/verify	—	✓
25	Officials	GET	/api/official/queue	—	✓
26	Officials	GET	/api/official/grievances/{id}	—	✓
27	Officials	POST	/api/official/grievances/{id}/actions	—	✓
28	Officials	GET	/api/official/metrics	—	✓
29	Mock	POST	/mock/municipal/{key}/complaints	—	—

Test Warning

 **Starlette Deprecation Warning**

StarletteDeprecationWarning: Using httpx with starlette.testclient is deprecated; install httpx2 instead.

This is a known Starlette deprecation warning about the `TestClient`'s HTTP backend. It does **not** affect test validity or results. No tests failed due to this warning.

User Feedback & Planned Improvements (Sprint 2 Roadmap)

Feedback Sessions Summary (28 July – 1 August 2026)

Between **28 July – 1 August 2026**, structured feedback sessions were conducted with representatives from each of our three primary user groups. Participants were walked through the live working prototype; their observations are documented below:

User Group 1 — Citizen (Web Portal)		VERIFIED PROTOTYPE
User Profile	Resident of Varanasi, aged 28, comfortable with smartphones and English/Hindi. Experienced with CPGRAMS; frustrated by complexity. Tested 4-step complaint wizard on mobile browser.	
Feedback Summary	Filing experience was smooth; appreciated no app installation required. Liked voice notes on Step 2 ("Describe issue") and map location picker on Step 1.	
Key Concerns	1. Dashboard Lacks Status Clarity: Complaints displayed technical status strings (<code>awaiting_confirmation</code> , <code>processing</code>). Requested plain-language descriptions and direct action buttons (e.g., "Your complaint is being analysed" or Confirm & Submit in dashboard row). 2. No PDF Receipt on Confirmation Screen: Success screen only showed ticket ID and duplicate count. Expected a direct Download / Share PDF Receipt option on the confirmation screen.	
Conclusion	Navigational and surfacing issues — core data/logic are correct, UI needs to be citizen-friendly.	
User Group 2 — Government Official (Triage Officer)		VERIFIED CONSOLE
User Profile	Mid-level civic officer familiar with internal ticketing tools. Tested Official Console (triage desk) using demo credentials, reviewing queue, evidence, and taking action.	
Feedback Summary	Triage queue layout well-organised; appreciated AI auto-populating category, summary, department, and confidence scores.	
Key Concerns	1. No Filter or Sort on Triage Queue: Queue displays pending complaints in arrival order. As volume grows, officers need department filtering (e.g., "Public Works cases") and safety priority sorting to avoid burying critical issues. 2. Clarification Request Flow Unclear: "Ask citizen" button exists, but queue cards lack a visual badge showing items awaiting citizen response versus fresh submissions.	
Conclusion	Focuses on queue management at scale — critical features as complaint volume increases.	
User Group 3 — WhatsApp-only User		VERIFIED BOT FLOW
User Profile	55-year-old resident of semi-urban ward using basic Android phone, comfortable only with WhatsApp and Hindi. Filed complaint entirely via WhatsApp bot.	
Feedback Summary	Successfully filed complaint using only WhatsApp (location sharing, Hindi voice note, ticket ID receipt) without assistance. Core flow validated end-to-end.	
Key Concerns	1. Reverse-OTP Verification Step Confusing: Linking required sending code (e.g. <code>JS-AB23CD</code>) back to bot. Expected auto-verification. Needs explicit instructions and an "Open WhatsApp →" deep-link button. 2. No Status Update on WhatsApp After Filing: After receiving ticket ID receipt, user heard nothing further. Needs proactive outbound messages (e.g. "Your complaint GRV-XXXX has been reviewed") to build trust.	
Conclusion	WhatsApp flow works end-to-end; needs onboarding copy refinement and proactive status updates.	

Plan for Sprint 2 (Milestone 4 Roadmap: 3 August – 12 August 2026)

#	FEATURE / IMPROVEMENT	TARGET USER GROUP	IMPLEMENTATION PLAN	OWNER	TARGET DATE
1	Human-Readable Status & Action Prompts	Citizen (Web Portal)	Replace raw status strings with plain-language text. Add visible "Review & Submit" button in dashboard row for <code>awaiting_confirmation</code> tickets.	Veer & Parkhi	8 Aug 2026
2	PDF Receipt Download on Confirmation	Citizen (Web Portal)	Add "Download Receipt" button to post-filing success screen when PDF is ready, or status indicator if generating.	Veer & Parkhi	8 Aug 2026

#	FEATURE / IMPROVEMENT	TARGET USER GROUP	IMPLEMENTATION PLAN	OWNER	TARGET DATE
3	Filter & Sort Controls on Triage Queue	Government Official	Implement department filtering, safety-level filtering, and arrival vs. priority sorting via query parameters on queue endpoint.	Tushar & Sarthak	10 Aug 2026
4	"Awaiting Citizen Response" Queue Badge	Government Official	Wire backend <code>review_status</code> field into frontend triage queue cards to display visual status badges.	Parkhi & Amit	10 Aug 2026
5	Improved Reverse-OTP Onboarding & Deep Link	WhatsApp User	Update onboarding prompt with clear instructions and prominent "Open WhatsApp →" deep-link button (<code>wa.me</code> URL).	Veer & Parkhi	6 Aug 2026
6	Proactive WhatsApp Status Notifications	WhatsApp User	Trigger automated outbound WhatsApp messages on status transitions (<code>processing</code> → <code>awaiting_confirmation</code> → <code>submitted</code>) using existing dispatch pipeline.	Tushar & Amit	12 Aug 2026

 Sprint 2 Readiness

All six planned improvements build upon foundational components already partially implemented in the Sprint 1 codebase. The final Sprint 2 (Milestone 4) delivery deadline is **12 August 2026**.